

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

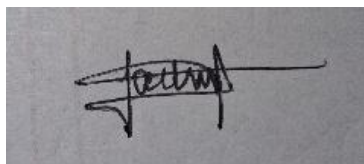
QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyze	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

I J a s h w a n t h . G . R (1 9 2 5 2 4 3 8 4) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate; correct final	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations

	answer			
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50),  
    HOD VARCHAR(50)  
);
```

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT,  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);
```

```
INSERT INTO Department VALUES  
(1,'CSE','Dr. Kumar'),  
(2,'ECE','Dr. Priya'),  
(3,'MECH','Dr. Raj');
```

```
INSERT INTO Student VALUES  
(101,'Arun',20,1),  
(102,'Priya',19,2),  
(103,'Rahul',21,1),  
(104,'Sneha',20,3),  
(105,'Kiran',22,2);
```

Question 1

Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.

1. INNER JOIN

```
SELECT S.StudentID,  
       S.Name,  
       S.Age,  
       D.DeptName  
FROM Student S  
INNER JOIN Department D  
ON S.DeptID = D.DeptID;
```

OUTPUT

StudentID	Name	Age	DeptName
101	Arun	20	CSE
102	Priya	19	ECE
103	Rahul	21	CSE
104	Sneha	20	MECH
105	Kiran	22	ECE

Explanation:

- SELECT retrieves the Student ID, Name, Age, and Department Name.
- FROM Student S selects the Student table and gives it the alias **S**.
- INNER JOIN Department D joins the Department table using the alias **D**.
- ON S.DeptID = D.DeptID matches each student with the corresponding department based on DeptID.
- The query returns only those records where a matching department exists for a student.

2.LEFT JOIN

```
SELECT S.StudentID,  
       S.Name,  
       D.DeptName  
FROM Student S  
LEFT JOIN Department D  
ON S.DeptID = D.DeptID;
```

OUTPUT:

StudentID	Name	DeptName
101	Arun	CSE
102	Priya	ECE
103	Rahul	CSE
104	Sneha	MECH
105	Kiran	ECE

Explanation

- LEFT JOIN returns all records from the Student table.
- If a student's DeptID matches a department, the department name is displayed.
- If no matching department exists, the department columns will show NULL.
- It is useful when you want to display every student, even if department information is missing.

3. RIGHT JOIN

```
SELECT D.DeptID,  
       D.DeptName,
```

```
S.Name
FROM Student S
RIGHT JOIN Department D
ON S.DeptID = D.DeptID;
```

OUTPUT:

DeptID	DeptName	Name
1	CSE	Arun
2	ECE	Priya
1	CSE	Rahul
3	MECH	Sneha
2	ECE	Kiran

Explanation

- RIGHT JOIN returns all records from the Department table.
- Matching student details are displayed if available.
- If a department has no students, the student columns will contain NULL.
- It is useful for finding departments without any students.

4. JOIN with WHERE Clause

```
SELECT S.StudentID,
       S.Name,
       D.DeptName
FROM Student S
JOIN Department D
ON S.DeptID = D.DeptID
```

WHERE D.DeptName = 'CSE';

OUTPUT:

StudentID	Name	DeptName
101	Arun	CSE
103	Rahul	CSE

Explanation

- The JOIN combines the Student and Department tables.
- The WHERE clause filters the results to display only students belonging to the CSE department.
- This query is useful when information from a specific department is required.

5. JOIN with ORDER BY

```
SELECT S.StudentID,  
       S.Name,  
       D.DeptName  
FROM Student S  
JOIN Department D  
ON S.DeptID = D.DeptID  
ORDER BY S.Name;
```

OUTPUT:

StudentID	Name ▲ 1	DeptName
101	Arun	CSE
105	Kiran	ECE
102	Priya	ECE
103	Rahul	CSE
104	Sneha	MECH

Explanation

- The JOIN combines the Student and Department tables.
- ORDER BY S.Name sorts the results in ascending alphabetical order based on the student name.
- This improves the readability of the output.

6. JOIN with COUNT()

```
SELECT D.DeptName,  
       COUNT(S.StudentID) AS TotalStudents  
FROM Department D  
JOIN Student S  
ON D.DeptID = S.DeptID  
GROUP BY D.DeptName;
```

OUTPUT

DeptName	TotalStudents
CSE	2
ECE	2
MECH	1

Explanation

- COUNT(S.StudentID) counts the number of students in each department.
- GROUP BY D.DeptName groups the records department-wise.
- The query displays the total number of students in every department.

7. JOIN with AVG()

```
SELECT D.DeptName,  
       AVG(S.Age) AS AverageAge  
FROM Student S  
JOIN Department D  
ON S.DeptID = D.DeptID  
GROUP BY D.DeptName;
```

OUTPUT:

DeptName	AverageAge
CSE	20.5000
ECE	20.5000
MECH	20.0000

Explanation

- AVG(S.Age) calculates the average age of students in each department.
- GROUP BY D.DeptName groups students according to their department.
- The result shows the average age for each department, which is useful for statistical analysis.

QUESTION 2

Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.

RELATIONAL ALGEBRA

$\pi(\text{StudentID}, \text{Name}, \text{DeptName})$

$(\sigma \text{ DeptName} = \text{'CSE'})$

$(\text{Student} \bowtie \text{Department})$

Explanation

- The JOIN combines the Student and Department tables.
- The WHERE clause filters only the CSE department.
- Relational Algebra performs the same operation using **Selection (σ)**, **Join ($\bowtie$)** and **Projection (π)**.

QUESTION 3

Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.

Retrival

```
SELECT S.StudentID,  
       S.Name,  
       S.Age,  
       D.DeptName  
FROM Student S  
JOIN Department D  
ON S.DeptID = D.DeptID;
```

OUTPUT

StudentID	Name	Age	DeptName
101	Arun	20	CSE
102	Priya	19	ECE
103	Rahul	21	CSE
104	Sneha	20	MECH
105	Kiran	22	ECE

Explanation

- The JOIN combines the Student and Department tables using DeptID.
- It displays each student's ID, name, age, and department name.

Update Student Age:

```
UPDATE Student  
SET Age = 21  
WHERE StudentID = 102;
```

Change Student Department:

```
UPDATE Student  
SET DeptID = 1  
WHERE StudentID = 105;
```

Retrieve Students from the CSE Department:

```
SELECT Name  
FROM Student  
WHERE DeptID =  
(  
    SELECT DeptID  
    FROM Department  
    WHERE DeptName = 'CSE'  
);
```

OUTPUT:

Name
Arun
Rahul
Kiran

QUESTION 4

Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.

```
SELECT Name
FROM Student
WHERE DeptID =
(
SELECT DeptID
FROM Department
WHERE DeptName='ECE'
);
```

OUTPUT:

Name
Priya

Explanation

- The inner query first finds the Department ID of ECE.
- The outer query uses that value to retrieve students from the ECE department.

QUESTION 5

Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency.

Using JOIN:

```
SELECT S.Name,D.DeptName  
FROM Student S  
JOIN Department D  
ON S.DeptID=D.DeptID;
```

OUTPUT:

Name	DeptName
Arun	CSE
Priya	ECE
Rahul	CSE
Sneha	MECH
Kiran	CSE

Using Subquery:

```
SELECT Name  
FROM Student  
WHERE DeptID=  
(  
SELECT DeptID  
FROM Department  
WHERE DeptName='CSE'  
);
```

OUTPUT

Name
Arun
Rahul
Kiran

Comparison

JOIN	Subquery
Faster	Slightly slower
Best for multiple tables	Best for filtering
Easy to retrieve related data	Easy to understand

QUESTION 6

Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.

```
CREATE VIEW Student_View AS
```

```
SELECT StudentID,
```

```
    Name,
```

```
    DeptID
```

```
FROM Student;
```

Retrieve data:

```
SELECT * FROM Student_View;
```

OUTPUT:

StudentID	Name	DeptID
101	Arun	1
102	Priya	2
103	Rahul	1
104	Sneha	3
105	Kiran	1

Explanation

- A View stores a query as a virtual table.
- It hides unnecessary columns and improves security.
- Users can access only the required information.

QUESTION 7

Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

Relational Algebra:

$\pi(\text{Name})$

$(\sigma_{\text{DeptID}=1}(\text{Student}))$

SQL Query

```
SELECT Name
FROM Student
WHERE DeptID=1;
```


OUTPUT:

Name

Arun

Rahul

Kiran

Explanation

- Projection (π) selects only the Name column.
- Selection (σ) filters students whose DeptID is 1.
- SQL performs the same operation using SELECT and WHERE.

QUESTION 8:

Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements

Inefficient Query

```
SELECT *  
FROM Student  
WHERE DeptID IN  
(  
SELECT DeptID  
FROM Department  
WHERE DeptName='CSE'  
);
```

Optimized Query

```
SELECT S.StudentID,  
       S.Name,  
       D.DeptName  
FROM Student S  
JOIN Department D  
ON S.DeptID=D.DeptID  
WHERE D.DeptName='CSE';
```

OUTPUT:

StudentID	Name	DeptName
101	Arun	CSE
103	Rahul	CSE
105	Kiran	CSE

Explanation

- The first query uses a nested subquery.
- The optimized query uses a JOIN.
- JOIN is generally faster because it reduces unnecessary nested execution and can make better use of indexes.

QUESTION 9

Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.

```
SELECT D.DeptName,  
       COUNT(S.StudentID) AS TotalStudents  
FROM Department D  
JOIN Student S
```

ON D.DeptID=S.DeptID

GROUP BY D.DeptName;

OUTPUT:

DeptName	TotalStudents
CSE	3
ECE	1
MECH	1

Explanation

- COUNT() counts the number of students.
- GROUP BY groups students according to their department.
- The result displays the total number of students in each department.

QUESTION 10:

Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.

Using JOIN

SELECT S.StudentID,

S.Name,

D.DeptName

FROM Student S

JOIN Department D

ON S.DeptID = D.DeptID;

OUTPUT:

StudentID	Name	DeptName
101	Arun	CSE
102	Priya	ECE
103	Rahul	CSE
104	Sneha	MECH
105	Kiran	CSE

Using SUBQUERY

```
SELECT Name
FROM Student
WHERE DeptID =
(
    SELECT DeptID
    FROM Department
    WHERE DeptName = 'CSE'
);
```

OUTPUT

Name
Arun
Rahul
Kiran

Using VIEW

```
CREATE VIEW Student_Department AS
```

```
SELECT S.StudentID,
```

```
       S.Name,
```

```
       D.DeptName
```

```
FROM Student S
```

```
JOIN Department D
```

```
ON S.DeptID = D.DeptID;
```

```
SELECT * FROM Student_Department;
```

OUTPUT:

StudentID	Name	DeptName
101	Arun	CSE
102	Priya	ECE
103	Rahul	CSE
104	Sneha	MECH
105	Kiran	CSE

CONCLUSION

- JOIN is preferred for retrieving related data from multiple tables because it is efficient and easy to understand.
- SUBQUERY is suitable when one query depends on the result of another query.
- VIEW is best for security, data abstraction, and simplifying frequently used queries.

